Busses

M. Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



1

Introduction

A bus is a structure that interconnects two or more devices.

A bus is a *shared* structure: the values a device writes on the bus are accessible to all other connected devices.

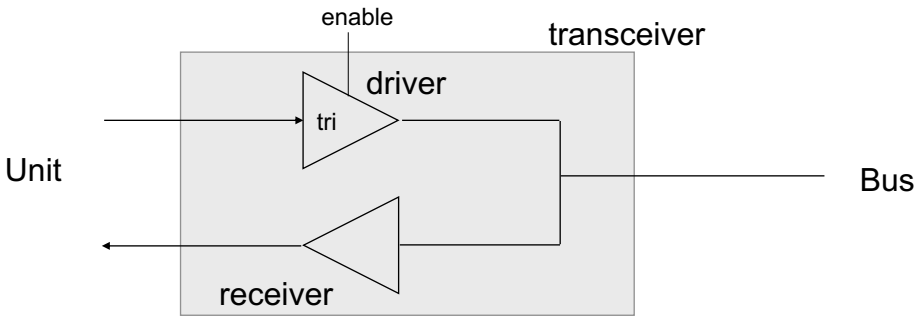
2

Bus interface

The units connected to the bus use two types of interfaces:

- **driver to drive bus lines (*tri-state driver*)**
- **receiver to read the values from the bus.**

The two devices are often grouped into a single entity named *transceiver*.



3

Bus implementation

You can have busses at different levels:

- **Inside a single integrated circuit, for connecting different modules**
- **On a board, for connecting multiple integrated circuits**
- **In a system, for connecting multiple cards (*backplane bus*).**

4

Bus structure

A bus consists of 3 groups of signals:

- ***Data* signals: normally, they are in a multiple of 8; they can be bidirectional or unidirectional (in this case you need a double number of lines);**
- ***Address* signals: they identify the slave the master wants to communicate with (and which part of the slave it is involved);**
- ***Control* signals: they provide information about status, timing, type (of bus data).**

5

Multiplexed busses

In some cases, the same lines carry different signals at different times.

In this way

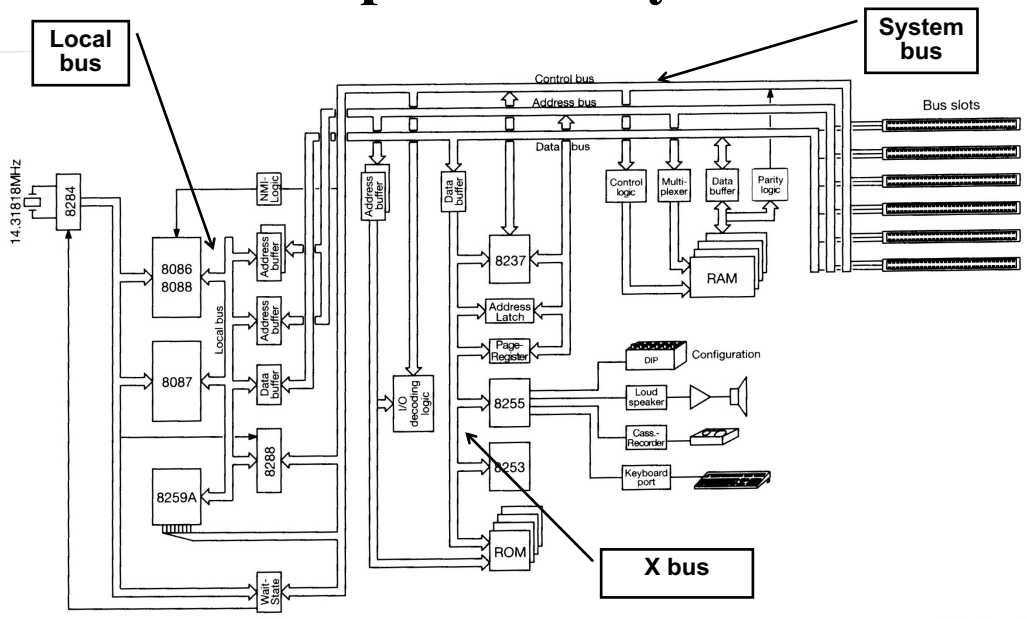
- **the number of bus lines is reduced**
- **Some additional circuitry is required.**

<u>Example</u>

In some buses the data lines and the address lines are multiplexed, and depending on the times the same lines carry data or address signals.

6

Example: 8086 system



Standard

In order to facilitate the development of compatible devices at the bus level, standards have been introduced that describe the bus characteristics at different levels

- ***mechanical*** **(for example, for backplane buses the number of lines, the type of connectors, the size of the cards)**
- ***electrical*** **(for example, the values of the voltages and reference currents)**
- ***logical / functional*** **(for example, the meaning of the various signals and the sequence of values that they must assume to perform each operation).**

Examples of standards are Amba, Multibus, PCI, VME, EISA, Futurebus +, SCSI.

8

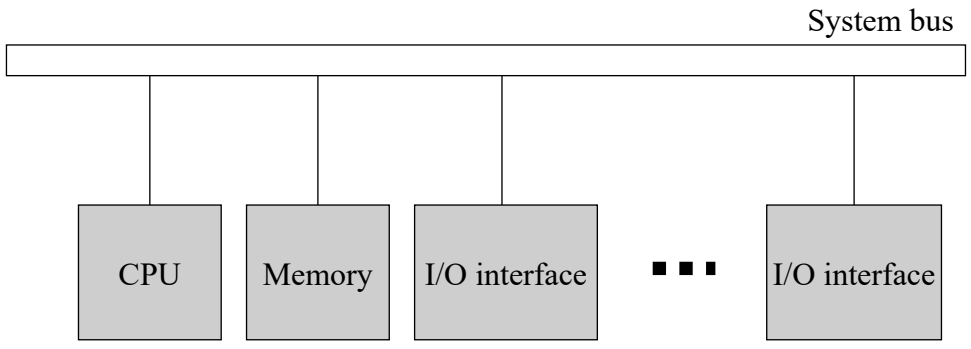
Bus architectures

You can have 2 types of bus architectures:

- ***single* bus: it is the simplest configuration**
- ***multiple* bus: it is useful when high performance is desired, or when devices of different classes have to be connected, with different characteristics.**

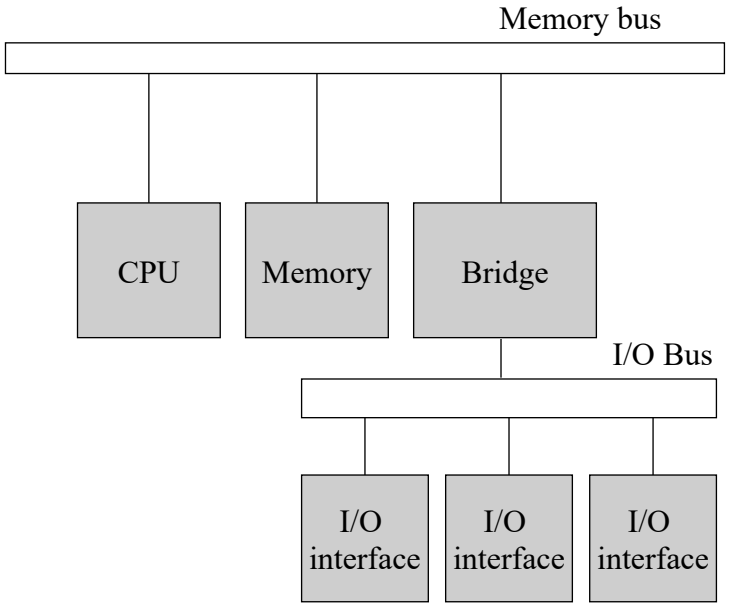
9

Single bus



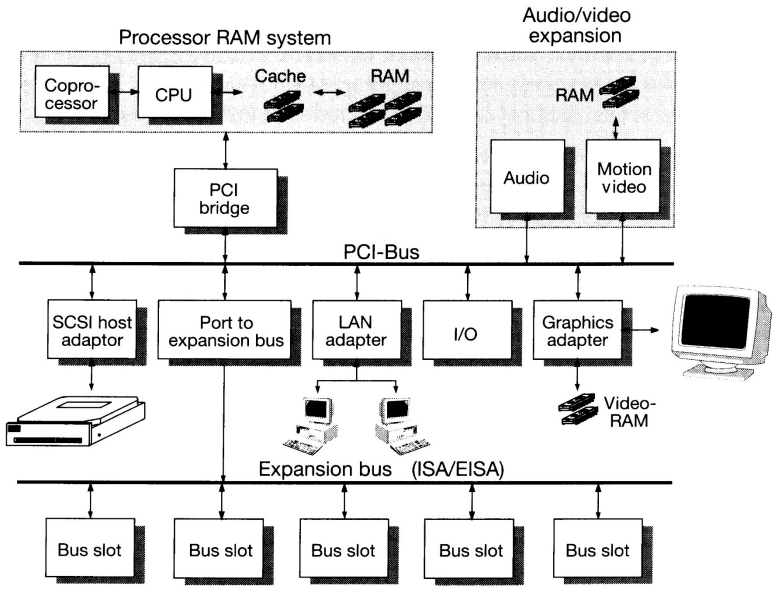
10

Multiple bus (example)



11

Example



12

Memory controller

In many systems there is a memory controller that deals with

- **interfacing the processor with the various memory banks**
- **handling some critical situations (such as memory errors)**
- **activating any timing signals.**

13

Master and slave modules

In a bus system, the connected units are of 2 types:

- ***Master unit*: it starts each data transfer procedure and chooses the slave to communicate with**
  - **In simpler systems there is only one master unit, which coincides with the CPU**
  - **In more complex systems there are more master units, and the master unit changes depending on the time**
- ***Slave unit*: it responds to the master unit commands. Memories and peripheral interfaces are slave units.**

14

Problems

Using a bus system asks for the solution of two major problems:

- **Defining the timing of bus operations**
- **Devising a mechanism to manage possible conflicts in the access to the bus.**

15

Timing

How can the various units connected to a bus know when to read or write from / to the bus?

Possible solutions belong to two types:

- **synchronous busses**
- **asynchronous busses.**

The specifications for a bus include, among the other aspects, the description of the protocol that the signals must follow, as well as the time constraints to be respected.

16

Synchronous busses

- **The source and destination units use the same clock signal, which is part of the bus itself; alternatively, the 2 units may have separate clocks but at the same frequency, and periodically exchange synchronization signals.**
- **The clock frequency is set by the slower device.**
- **Each data unit is transferred during a predetermined time period (normally a clock period).**
- **The mechanism works well on reduced distances.**

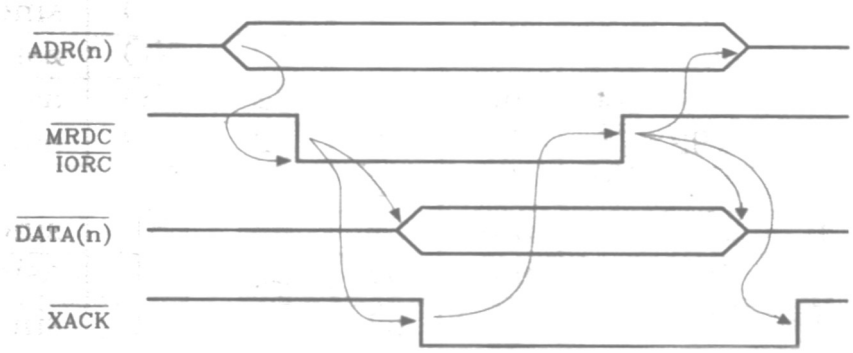
17

Asynchronous busses

- **Each communication operation can have its own speed, determined by appropriate control signals that accompany the data signals and implement the so-called *handshaking.***
- **In this way we get the maximum flexibility at the expense of a greater complexity of the protocol.**

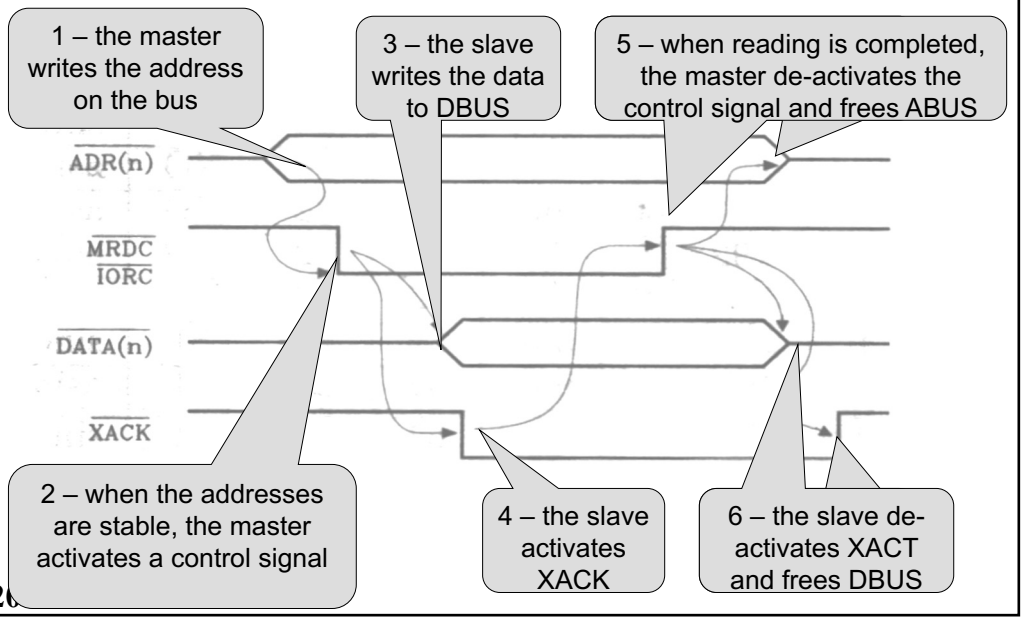
18

Multibus I: reading cycle



19

Multibus I: reading cycle



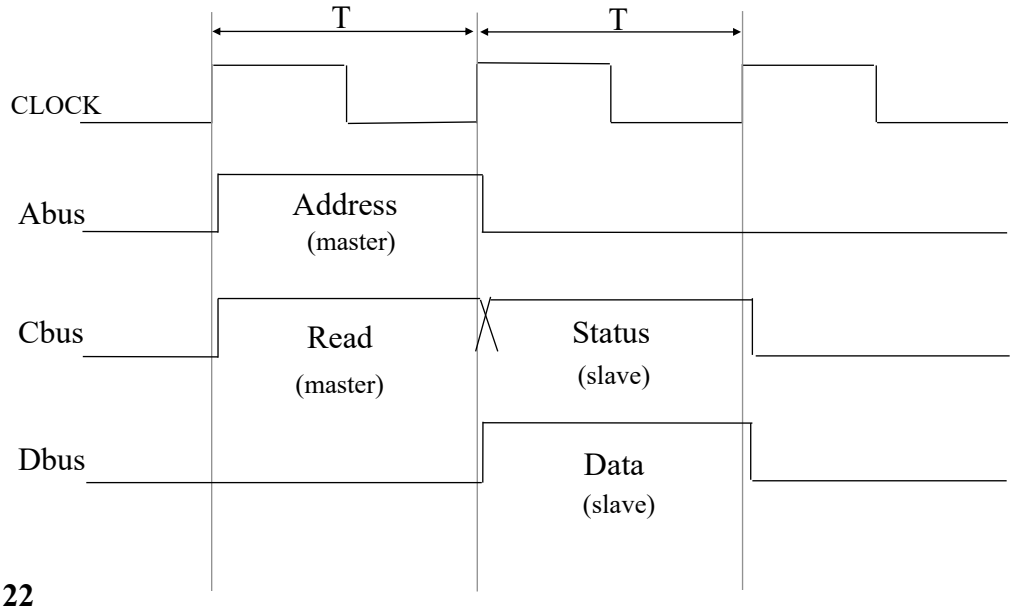
20

Wait cycles

- **In the case of an synchronous bus, there is a mechanism for introducing occasionally additional cycles (called *wait cycles*) that allow the slave to get more time to perform the required operation.**
- **The mechanism uses a special signal (READY) indicating the need to add further cycles.**

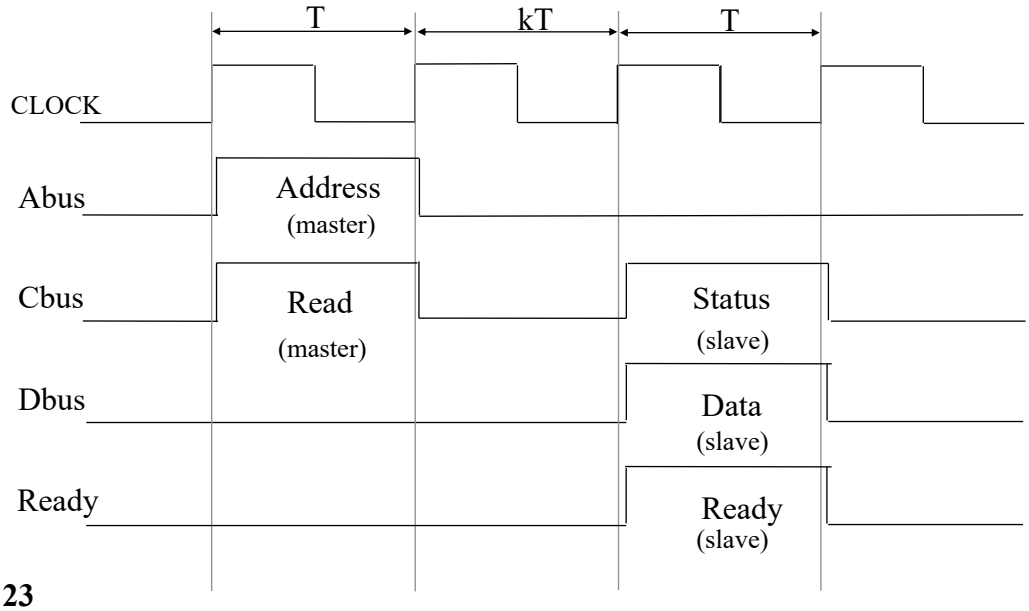
21

Read operation without wait cycles



22

Read operation with wait cycles



23

Bus arbitration

- **At each time, only one device can act as the bus master**
- **The arbitration mechanism of the bus comes into operation when two or more units simultaneously request to become the master of the bus: the mechanism must then designate the new master unit**
- **Arbitration can take place in two possible ways**
  - ***centralized*: there is an arbiter**
  - ***distributed*: each module contains the logic necessary to implement an arbitration mechanism that allows to define the new master.**

24

Distributed arbitration: the SCSI bus

- **The SCSI bus includes 8 lines `DB(0)`,…, `DB(7)` which are used for both data transfer and arbitration**
- **During arbitration, each line is associated to a module: line `DB(7)` has the maximum priority**
- **When the line `BSY` becomes inactive, all units (at most 8) which want to access the bus activate their own line `DB(i)`.**
- **Then, all units observe the values on lines `DB(0)`, …, `DB(7)`, and the unit with the maximum priority wins the context; the others wait until `BSY` is deactivated again.**

25

Centralized arbitration

There are 3 centralized arbitration mechanisms:

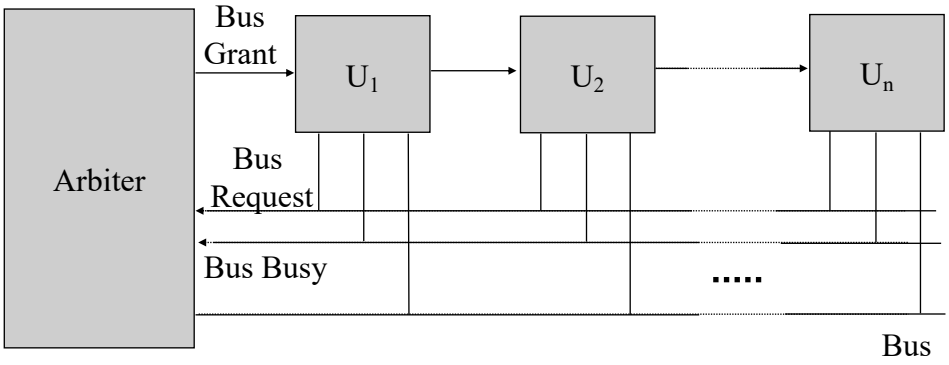
- ***Daisy Chaining***
- ***Polling***
- ***Independent requests*.**

They differ for:

- **number of control lines required**
- **response speed of the bus controller**
- **flexibility in managing priorities**
- **Fault Tolerance.**

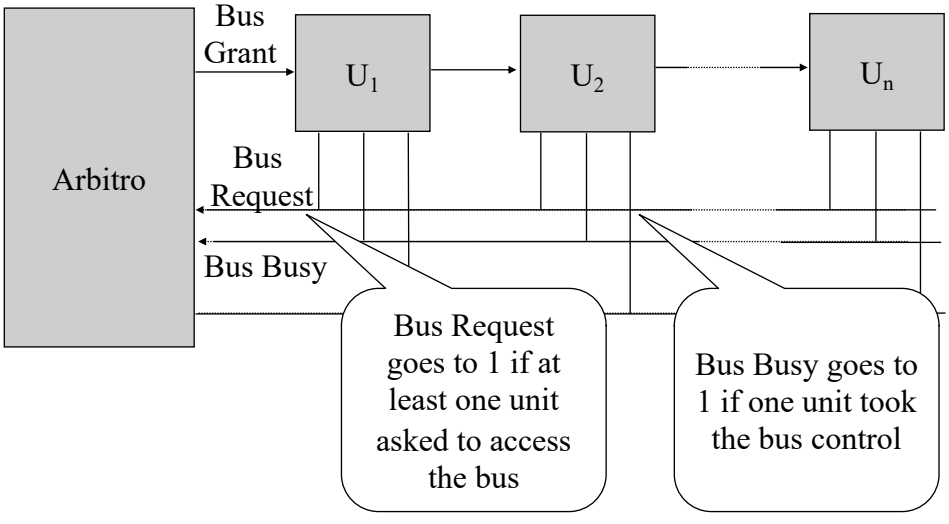
26

Daisy Chaining



27

Daisy Chaining



28

Daisy Chaining: behavior

- **One unit requests the bus (BUS REQUEST) when the bus is free (BUS BUSY)**
- **The arbiter activates the BUS GRANT signal**
- **Each unit receives the BUS GRANT:**
  - **if it required the bus: it activates BUS BUSY and takes the control of the bus**
  - **if it did not require the bus: it activates BUS GRANT towards the downstream unit.**

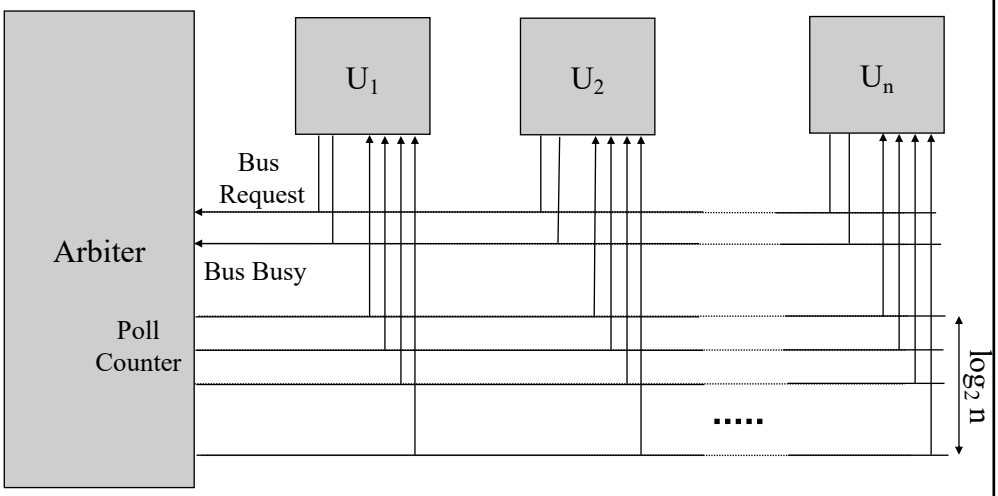
29

Daisy Chaining: features

+ **It requires only 3 control signals**
- **It does not allow you to change the unit priorities**
- **It is not suitable for systems with a high number of units**
- **It is not tolerant to failures.**

30

Polling



31

Polling

- **One unit requests the bus (BUS REQUEST) when the bus is free (BUS BUSY)**
- **The arbiter polls all the connected units by writing the identifier of each unit, in sequence, on the POLL COUNTER lines**
- **When one unit is addressed**
  - **if it had requested the bus it takes the control of the bus, and activates the BUS BUSY signal; at this point the arbiter interrupts the poll**
  - **if not, it does nothing, and the arbiter passes to the next unit.**

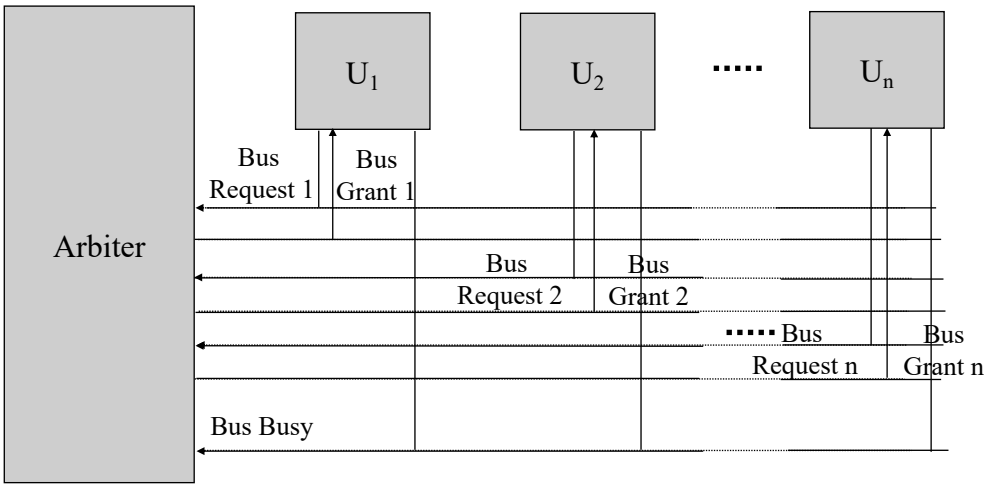
32

Polling: features

- It needs `2+log(n)` control signals to manage `n` units
+ The priority of the different units can be changed by modifying the poll sequence
+ The system can tolerate any failure in a unit.

33

Independent requests



34

Independent requests: behavior

- The i-th unit, when it wants to access the bus
  - It waits until the bus is free (observing BUS BUSY)
  - It sends a request to the arbiter (activating BUS REQUEST i)
- The arbiter manages all requests, and grants the bus to the unit with maximum priority (BUS GRANT j) among those that made the request
- Unit j takes the control of the bus (and activates BUS BUSY).

35

Independent requests: features

- It needs `2*n+1` control signals to manage `n` units

+ The priorities of the different units depend on the mechanism implemented by the arbiter

+ The system can tolerate a possible fault in a unit.

36